

RCC Onboarding - Part 2

Concepts and reproducible workflows with Miniforge, Bioconda, Snakemake, Slurm, statistics, and DNA sequence analysis

IKIM RCC documentation proposal

11 July 2026

Contents

Purpose, learning goals, and essential vocabulary

Information that administrators must complete

Why workflows are different from command lists

Project storage and directory design

Software environments: Miniforge, Conda, and Mamba

Channels: conda-forge and Bioconda

Snakemake control and rule-specific environments

Connection lifetimes and tmux

Statistical workflow and interpretation boundaries

DNA sequence workflow and file formats

Incremental execution

Rule-level Slurm resources

Logs and layered failure handling

Reproducibility and provenance

Git without data leakage

Common beginner mistakes

Completion checklist

Next steps

References for maintainers

Purpose and learning goals

Part 2 is for users who have completed Part 1 and can already connect to the RCC submission host, transfer data, and submit a basic Slurm batch job.

Part 2 moves from “running a command” to “designing an analysis that can be repeated, reviewed, resumed, and executed safely on shared computing infrastructure.” The emphasis is not only on syntax. Each tool is introduced as one component of a reproducible system.

By the end of Part 2, you will be able to:

create isolated software environments with Miniforge and Mamba;
 obtain scientific software from conda-forge and Bioconda;
 organise an analysis as a reproducible Snakemake project;
 use Snakemake to submit individual analysis steps to Slurm;
 run a small statistical workflow that produces tables and a figure;
 run a minimal paired-end DNA sequence workflow that performs read quality control and mapping; and
 record the software, parameters, logs, and resource use needed to reproduce an analysis.

This tutorial is deliberately prescriptive. Use the directory names and commands as written for the exercises. Adapt them only after both examples work.

Essential vocabulary for Part 2

Package: an installable unit of software, such as Python, pandas, fastp, or samtools.

Dependency: another package required by a package.

Environment: an isolated directory containing a defined set of packages.

Channel: a repository from which Conda-compatible packages are obtained.

Workflow: a connected set of analysis steps with declared inputs and outputs.

Rule: a Snakemake definition of one transformation from input files to output files.

Target: the file or collection of files requested from the workflow.

DAG: directed acyclic graph, the dependency structure connecting rules.

Provenance: evidence describing which data, software, parameters, and commands produced a result.

Reference data: a curated sequence, annotation, or database used as a basis for comparison.

Intermediate file: a file produced by one rule and consumed by another.

The five layers of a reproducible RCC analysis

Scientific question: what is being estimated or tested?

Data model: what does each row, column, sample, and file represent?

Workflow model: which transformations create the final outputs?

Software model: which exact packages and versions are required by each transformation?

Execution model: which Slurm resources and storage locations are used?

A workflow can execute successfully and still be scientifically invalid. Snakemake and Slurm provide execution discipline; they do not choose an appropriate statistical test, reference genome, quality threshold, or biological interpretation.

Information that administrators must complete

Administrator action required. Replace every value marked **ADMIN** before publication. Users must not have to infer cluster policy from examples.

Supported Miniforge source: [ADMIN: RCC-provided installer, module, or approved download procedure]

Miniforge installation location: [ADMIN: recommended user path or shared installation]

Conda package and environment location: [ADMIN: approved location and quota guidance]

RCC Snakemake profile: [ADMIN: exact profile name and invocation]

Required Slurm account: [ADMIN: state whether --account is required]

Default CPU partition: [ADMIN: exact partition or state that the profile chooses it]

Maximum beginner workflow concurrency: [ADMIN: recommended --jobs value]

Internet access policy: [ADMIN: explain where package downloads are permitted]

Supported Snakemake version: [ADMIN: minimum or pinned version]

RCC support contact: [ADMIN: support email or ticket address]

The examples below use RCC_PROFILE as a placeholder for the supported Snakemake profile. Replace it with the profile published by RCC.

1. The workflow model

Why a workflow is more than a script

A script describes a sequence of commands. A workflow also records dependencies between files, knows which outputs are missing or outdated, can run independent steps in parallel, associates software and resources with each step, and stops downstream work when an upstream step fails.

This matters because real analyses are rarely executed only once. Input samples arrive, parameters change, a failed step must be resumed, software versions evolve, and reviewers ask how a figure was generated. A workflow turns those events into controlled recomputation instead of manual command history.

A reproducible analysis separates four concerns:

data: immutable inputs and generated outputs;

code: scripts and workflow rules;

configuration: sample names and analysis parameters;

software: explicit package environments.

Snakemake connects these elements as a directed acyclic graph. “Directed” means that data flow from inputs toward outputs. “Acyclic” means that the dependency chain cannot loop back to itself. Each rule declares the files it consumes, the files it creates, the command or script it runs, the software environment it needs, and the resources it should request.

Snakemake works backward from requested targets. If a target already exists and its inputs and workflow definition indicate that it is current, the rule may not run again. If an input or rule changes, affected downstream files can be rebuilt. This is called incremental execution.

```

input data
  |
  v
quality control -----> QC report
  |
  v
analysis -----> result tables
  |
  v
visualisation -----> figures and final report

```

Snakemake determines which rules are required for the requested final files. On RCC, the Snakemake process coordinates the workflow on a submission host; the actual rules should run as Slurm jobs on compute nodes.

Do not run the analysis rules directly on the submission host. A dry run and workflow planning are appropriate there. Computation must be sent through the supported RCC Snakemake/Slurm profile.

2. Decide where the project belongs

Why directory design is part of reproducibility

A workflow uses paths as part of its data model. Mixing raw inputs, hand-edited results, logs, temporary files, and code in one directory makes it difficult to determine what can be regenerated and what must be preserved. A predictable project layout also makes collaboration, backup, access control, and support easier.

Use the project or group directory assigned by your coordinator. Do not place shared research data or complete workflow results permanently in your home directory.

The recommended separation is:

data/: exercise inputs or links to approved immutable inputs. Do not edit workflow inputs in place.

config/: sample sheets and analysis parameters. Review changes as code.

envs/: software environment definitions. Pin or constrain important versions.

workflow/: Snakefile and analysis scripts. Track these files in Git.

logs/: standard output and error from jobs. Inspect them; normally they can be regenerated.

results/: generated tables, figures, and processed data. Do not hand-edit them.

For this tutorial, define a work root. Replace the example with your approved path:

```
export RCC_WORK=/projects/PROJECT_NAME/YOUR_ASSIGNED_DIRECTORY/tutorials
mkdir -p "$RCC_WORK"
cd "$RCC_WORK"
```

Confirm the location:

```
pwd
ls -ld .
```

The directory must be visible from the submission hosts and compute nodes. Do not use /tmp, /local, or a node-local scratch directory as the workflow root unless RCC documentation explicitly describes a staging workflow.

3. Install or activate Miniforge

3.1 Why scientific software needs environment management

Scientific tools often require conflicting versions of Python, system libraries, compilers, and other programs. Installing everything into one shared location creates “dependency conflicts”: changing one tool can break another. Installing into the operating system also requires administrator access and makes an analysis difficult to reproduce elsewhere.

An environment isolates a compatible software set. Activating an environment changes the command search path so that the shell finds the programs inside that environment first. It does not create a virtual machine and it does not allocate compute resources.

3.2 Conda, Mamba, Miniforge, conda-forge, and Bioconda

These names describe different layers:

Conda is the package and environment-management format and command-line tool.

Mamba is a faster compatible solver and installer.

Miniforge is a small distribution that installs Conda/Mamba with conda-forge as its foundation.

conda-forge is a community channel containing general scientific and system packages.

Bioconda is a channel specialising in bioinformatics packages and built to work with conda-forge.

Miniforge is not the same as the commercial Anaconda distribution. Use the RCC-supported method so that licensing, storage, updates, and network access are consistent across the cluster.

3.3 What Miniforge provides

Miniforge is a minimal conda-forge distribution that provides conda and mamba. Environments created with it are isolated directories containing a specific collection of software packages and dependencies.

This tutorial uses Mamba because it resolves environments quickly while remaining compatible with Conda environment files.

3.4 Prefer the RCC-supported installation method

Follow **[ADMIN: supported Miniforge procedure]**. One of the following patterns will normally apply.

Pattern A: RCC provides Miniforge centrally

The administrator may provide a module or shell initialisation command. For example only:

```
module load miniforge
```

Do not copy this example unless RCC publishes it.

Pattern B: RCC provides an installer on shared storage

For example only:

```
bash /software/installers/Miniforge3-Linux-x86_64.sh -b -p "$HOME/.local/miniforge3"
```

Pattern C: RCC permits installation from the official release

Only use the URL and checksum published by RCC. Verify the installer before executing it. Do not copy an installer from another user or run an unverified script received by email.

3.5 Initialise the shell

If Miniforge was installed in `$HOME/.local/miniforge3`, initialise Bash once:

```
"$HOME/.local/miniforge3/bin/conda" init bash
```

Close the terminal, open a new VS Code terminal, and test:

```
conda --version
```

```
mamba --version
```

Disable automatic activation of the base environment:

```
conda config --set auto_activate_base false
```

Open another terminal and confirm that (base) is not added automatically to the prompt.

4. Configure conda-forge and Bioconda

What channels and channel priority mean

A package name may exist in more than one channel and may have been compiled against different libraries. Channel priority determines which repository is preferred. Strict priority reduces accidental mixing of incompatible builds and makes environment resolution more predictable.

Bioconda packages are built to work with conda-forge. Configure the channels once, in the published order, and use strict channel priority:

```
conda config --add channels bioconda
```

```
conda config --add channels conda-forge
```

```
conda config --set channel_priority strict
```

Inspect the effective configuration:

```
conda config --show channels
conda config --show channel_priority
```

The channel order should place conda-forge before bioconda, followed by any explicitly approved fallback. Strict priority reduces accidental mixing of incompatible builds.

For workflow-specific environments, prefer environment files that declare:

channels:

- conda-forge
- bioconda
- nodefaults

Not every environment needs Bioconda. Statistical Python environments should normally use only conda-forge and nodefaults.

5. Create the Snakemake control environment

Why the workflow engine and analysis tools are separated

Snakemake is the coordinator. pandas, fastp, minimap2, samtools, and other programs are workers used by individual rules. Keeping the coordinator in a small control environment avoids a large collection of unrelated dependencies and allows each rule to declare only the software it needs.

This separation also improves provenance. A future reader can see that one rule used a specific statistics environment while another used a specific bioinformatics environment.

The Snakemake executable belongs in its own control environment. Analysis tools belong in per-rule environments and must not be added to the Snakemake control environment.

Create the control environment:

```
mamba create --name snakemake \
  --channel conda-forge \
  --channel bioconda \
  snakemake \
  snakemake-storage-plugin-fs \
  snakemake-executor-plugin-slurm
```

Activate it and record the version:

```
conda activate snakemake
snakemake --version
```

Export a portable record:

```
conda env export --from-history --name snakemake > "$HOME/snakemake-control-environment.yml"
```

The `--from-history` export records the packages you explicitly requested rather than every platform-specific dependency.

If RCC supplies a pinned environment or wrapper command, use that instead. Do not update a centrally maintained environment yourself.

6. Use tmux for workflow drivers

The three lifetimes involved

A distributed workflow has three different lifetimes:

- the **SSH/VS Code connection** from your laptop, which may be interrupted;
- the **Snakemake driver** on a submission host, which plans and monitors jobs; and
- the **Slurm jobs** on compute nodes, which Slurm owns after submission.

A Snakemake driver may run longer than a VS Code connection. Start it in tmux so a network interruption does not terminate the workflow. tmux preserves the driver shell on the same submission host. Slurm independently preserves submitted compute jobs.

What tmux does not do

It does not request a compute node.

It does not make CPU-intensive work acceptable on the submission host.

It does not replace Slurm logs or Snakemake logs.

It does not automatically move between load-balanced submission hosts. You may need to reconnect to the same physical host before attaching.

Create a session:

```
tmux new -s rcc-tutorial
```

Inside tmux, record the physical submission host:

```
hostname
```

Detach without stopping the workflow by pressing Ctrl+B, then D.

Reconnect to the same physical host, if required by RCC load balancing, and attach:

```
tmux attach -t rcc-tutorial
```

List sessions:

```
tmux ls
```

7. Statistical analysis workflow

What this example teaches

The goal is not to prescribe a statistical method for real studies. The example demonstrates how a table moves through validation, analysis, and visualisation while every step has declared inputs, outputs, software, logs, and resources.

Understand the input table before analysing it

A rectangular data file is meaningful only when the units of observation are clear. In this exercise:

each row is one synthetic sample;

sample_id uniquely identifies the row;

group is a categorical explanatory variable; and

value is a numeric outcome.

Real analyses must also define missing-value codes, measurement units, inclusion criteria, repeated measurements, batch variables, and whether observations are independent.

What the statistical outputs mean

A **group summary** describes the observed data, for example count, mean, and standard deviation.

A **Welch two-sample t-test** compares group means without assuming equal variances. It still assumes independent observations and a suitable measurement scale.

A **p-value** measures compatibility with a null model under the test assumptions; it is not the probability that the scientific hypothesis is true.

A **figure** helps inspect distribution, outliers, and group overlap; it is not a replacement for a defined statistical model.

For real work, report effect estimates and uncertainty, check assumptions, account for study design and multiple testing, and obtain statistical review when appropriate.

This workflow reads a CSV table, validates it, calculates group summaries, performs a Welch two-sample t-test, and creates a box-and-jitter plot.

7.1 Create the project structure

```
cd "$RCC_WORK"
mkdir -p stats-demo/{config,data,envs,workflow/scripts,logs,results}
cd stats-demo
```

The final structure will be:

```
stats-demo/
├── config/
│   └── config.yaml
├── data/
│   └── measurements.csv
├── envs/
│   └── stats.yaml
├── workflow/
│   ├── Snakefile
│   └── scripts/
│       └── analyse.py
├── logs/
└── results/
```

7.2 Create the input table

Create data/measurements.csv:

```
sample,group,value
C01,control,10.1
C02,control,9.8
C03,control,10.4
C04,control,9.9
C05,control,10.2
C06,control,10.0
T01,treatment,11.4
T02,treatment,11.0
T03,treatment,11.8
T04,treatment,10.9
T05,treatment,11.5
T06,treatment,11.2
```

This is synthetic teaching data. Do not interpret it as a biological or clinical result.

7.3 Create the configuration

Create config/config.yaml:

```
input_table: data/measurements.csv
group_column: group
value_column: value
control_group: control
treatment_group: treatment
```

Keeping parameters in a configuration file makes changes visible and avoids embedding project-specific values throughout the code.

7.4 Define the statistical software environment

Create envs/stats.yaml:

```
channels:
- conda-forge
- nodefaults
dependencies:
- python=3.12
- pandas
- scipy
- matplotlib
```

Do not add packages interactively after the workflow works. Change the environment file and let Snakemake recreate the environment.

7.5 Create the analysis script

Create workflow/scripts/analyse.py:

```
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from scipy import stats

input_path = Path(snakemake.input.table)
summary_path = Path(snakemake.output.summary)
test_path = Path(snakemake.output.test)
figure_path = Path(snakemake.output.figure)

for path in (summary_path, test_path, figure_path):
    path.parent.mkdir(parents=True, exist_ok=True)

df = pd.read_csv(input_path)
required = {
    "sample",
    snakemake.params.group_column,
    snakemake.params.value_column,
}
missing = sorted(required.difference(df.columns))
if missing:
    raise ValueError(f"Missing required columns: {missing}")

if df["sample"].duplicated().any():
    duplicates = df.loc[df["sample"].duplicated(), "sample"].tolist()
    raise ValueError(f"Duplicate sample identifiers: {duplicates}")

value_column = snakemake.params.value_column
group_column = snakemake.params.group_column
control_name = snakemake.params.control_group
treatment_name = snakemake.params.treatment_group

df[value_column] = pd.to_numeric(df[value_column], errors="raise")

summary = (
    df.groupby(group_column, observed=True)[value_column]
    .agg(n="count", mean="mean", standard_deviation="std", median="median")
    .reset_index()
)
```

```

summary.to_csv(summary_path, sep="\t", index=False)

control = df.loc[df[group_column] == control_name, value_column]
treatment = df.loc[df[group_column] == treatment_name, value_column]
if len(control) < 2 or len(treatment) < 2:
    raise ValueError("Each comparison group requires at least two observations")

result = stats.ttest_ind(treatment, control, equal_var=False)
test = pd.DataFrame(
    {
        "comparison": [f"{treatment_name} versus {control_name}"],
        "method": ["Welch two-sample t-test"],
        "statistic": [result.statistic],
        "p_value": [result.pvalue],
        "mean_difference": [treatment.mean() - control.mean()],
    }
)
test.to_csv(test_path, sep="\t", index=False)

ordered_groups = [control_name, treatment_name]
values = [df.loc[df[group_column] == group, value_column] for group in ordered_groups]

fig, ax = plt.subplots(figsize=(7, 4.5))
ax.boxplot(values, tick_labels=ordered_groups, showmeans=True)
for position, group_values in enumerate(values, start=1):
    offsets = [position + (i - (len(group_values) - 1) / 2) * 0.025 for i in range(len(group_values))]
    ax.scatter(offsets, group_values, zorder=3)
ax.set_xlabel(group_column)
ax.set_ylabel(value_column)
ax.set_title("Synthetic RCC statistics tutorial")
fig.tight_layout()
fig.savefig(figure_path, dpi=160)
plt.close(fig)

```

The object named `snakemake` is provided automatically when the script is executed by a Snakemake script: directive.

7.6 Create the Snakefile

Create workflow/Snakefile:

configfile: "config/config.yaml"

```

rule all:
    input:
        "results/group-summary.tsv",
        "results/welch-test.tsv",
        "results/group-plot.png",

rule analyse_statistics:
    input:
        table=config["input_table"],
    output:
        summary="results/group-summary.tsv",
        test="results/welch-test.tsv",
        figure="results/group-plot.png",
    params:
        group_column=config["group_column"],
        value_column=config["value_column"],

```

```

    control_group=config["control_group"],
    treatment_group=config["treatment_group"],
threads: 1
resources:
    mem_mb=2000,
    runtime=10,
conda:
    "../envs/stats.yaml"
log:
    "logs/statistics.log"
script:
    "scripts/analyse.py"

```

The paths in outputs are relative to the project working directory from which Snakemake is invoked. The environment and script paths are relative to the Snakefile.

7.7 Check the workflow before running it

From the project root:

```

cd "$RCC_WORK/stats-demo"
conda activate snakemake
snakemake --snakefile workflow/Snakemake --dry-run --printshellcmds

```

A dry run builds the dependency graph but does not execute the statistical analysis.

Run a lint check:

```
snakemake --snakefile workflow/Snakemake --lint
```

Review warnings rather than suppressing them automatically.

7.8 Run through Slurm

Use the RCC-supported profile:

```

nice snakemake \
  --snakefile workflow/Snakemake \
  --profile RCC_PROFILE \
  --jobs 10 \
  --software-deployment-method conda \
  --rerun-incomplete \
  --printshellcmds

```

--jobs 10 is a concurrency ceiling, not a request for ten CPUs for every rule. Each rule declares its own threads and resources.

If the profile requires an explicit account or other site option, use the command published by RCC rather than inventing an option.

7.9 Inspect the results

```

column -t -s $'\t' results/group-summary.tsv
column -t -s $'\t' results/welch-test.tsv
ls -lh results/group-plot.png

```

Open results/group-plot.png in VS Code.

Check workflow status:

```
snakemake --snakefile workflow/Snakemake --summary
```

Check Slurm accounting for the jobs created during the run:

```

sacct --starttime today --user "$USER" \
  --format=JobID,JobName,State,Elapsed,AllocCPUS,ReqMem,MaxRSS,ExitCode

```

A result is not complete merely because a figure exists. Confirm that Snakemake reports all requested outputs as up to date and that the Slurm jobs completed with exit code zero.

8. Minimal DNA sequence-analysis workflow

The sequence-analysis data model

This exercise uses synthetic paired-end reads and a tiny synthetic reference. It is designed to teach file flow, not to make a biological claim.

Key file types:

FASTA (.fa, .fasta): reference sequences represented by names and nucleotide strings.

FASTQ (.fastq, often .fastq.gz): sequencing reads plus per-base quality scores.

Paired-end reads (R1 and R2): two reads originating from opposite ends of the same DNA fragment.

SAM/BAM: alignment records describing where reads map to a reference; BAM is the compressed binary form.

BAI: an index that allows software to access selected regions of a BAM efficiently.

QC report: metrics about read quality, trimming, alignment, or file content.

Why quality control and mapping are separate rules

Raw reads may contain adapters, low-quality bases, or other artefacts. fastp performs read-level quality control and trimming. minimap2 then compares the processed reads with the reference. samtools converts, sorts, indexes, and summarises the alignment records. MultiQC aggregates reports so that many samples can be reviewed consistently.

A high mapping percentage is not automatically a correct biological result. Interpretation depends on the reference, contamination, read length, repeats, taxonomic mixture, parameters, and experimental design.

This example creates a tiny synthetic reference and paired-end FASTQ dataset, performs read QC and trimming with fastp, maps reads with minimap2, creates a sorted/indexed BAM with samtools, and generates a MultiQC report.

The data are synthetic and intentionally tiny. The workflow structure, not the scientific result, is the lesson.

8.1 Create the project structure

```
cd "$RCC_WORK"
mkdir -p dna-demo/{config,data,envs,workflow/scripts,logs,results}
cd dna-demo
```

8.2 Create the data-generation script

Create workflow/scripts/make_demo_data.py:

```
from pathlib import Path
import gzip

reference = (
    "ACGTTGCAAGCTTAGCGATCGATGCTAGCTAGGCTAACCGTTAGCCTAGGATCCGATGCTAGCTA"
    "CGATCGTACGATGCTAGCATCGATGCTAGCATGCTAGCATCGATCGTAGCTAGCTAACGTA"
)

reference_path = Path(snakemake.output.reference)
r1_path = Path(snakemake.output.r1)
r2_path = Path(snakemake.output.r2)
for path in (reference_path, r1_path, r2_path):
```

```

path.parent.mkdir(parents=True, exist_ok=True)

reference_path.write_text(f">demo_reference\n{reference}\n", encoding="utf-8")

starts = [0, 8, 16, 24, 32, 40, 48, 56]
read_length = 50

def reverse_complement(sequence: str) -> str:
    table = str.maketrans("ACGT", "TGCA")
    return sequence.translate(table)[::-1]

with gzip.open(r1_path, "wt", encoding="utf-8") as r1, gzip.open(
    r2_path, "wt", encoding="utf-8"
) as r2:
    for index, start in enumerate(starts, start=1):
        forward = reference[start : start + read_length]
        reverse_start = start + 20
        reverse = reverse_complement(reference[reverse_start : reverse_start + read_length])
        quality1 = "I" * len(forward)
        quality2 = "I" * len(reverse)
        r1.write(f"@demo_{index}/1\n{forward}\n+\n{quality1}\n")
        r2.write(f"@demo_{index}/2\n{reverse}\n+\n{quality2}\n")

```

8.3 Define the environments

Create envs/python.yaml:

```

channels:
- conda-forge
- nodefaults
dependencies:
- python=3.12

```

Create envs/fastp.yaml:

```

channels:
- conda-forge
- bioconda
- nodefaults
dependencies:
- fastp

```

Create envs/mapping.yaml:

```

channels:
- conda-forge
- bioconda
- nodefaults
dependencies:
- minimap2
- samtools

```

Create envs/multiqc.yaml:

```

channels:
- conda-forge
- bioconda
- nodefaults
dependencies:
- multiqc

```

Separate environments reduce dependency conflicts and make the software used by each rule explicit.

8.4 Create the DNA Snakefile

Create workflow/Snakefile:

SAMPLE = "demo"

rule all:

input:

```
"results/mapping/demo.sorted.bam",
"results/mapping/demo.sorted.bam.bai",
"results/mapping/demo.flagstat.txt",
"results/multiqc/multiqc_report.html",
```

rule make_demo_data:

output:

```
reference="data/reference.fasta",
r1="data/demo_R1.fastq.gz",
r2="data/demo_R2.fastq.gz",
```

threads: 1

resources:

```
mem_mb=500,
runtime=5,
```

conda:

```
"../envs/python.yaml"
```

script:

```
"scripts/make_demo_data.py"
```

rule fastp:

input:

```
r1="data/demo_R1.fastq.gz",
r2="data/demo_R2.fastq.gz",
```

output:

```
r1="results/trimmed/demo_R1.fastq.gz",
r2="results/trimmed/demo_R2.fastq.gz",
html="results/qc/fastp.html",
json="results/qc/fastp.json",
```

log:

```
"logs/fastp.log"
```

threads: 2

resources:

```
mem_mb=2000,
runtime=15,
```

conda:

```
"../envs/fastp.yaml"
```

shell:

```
"""
```

```
fastp \
--in1 {input.r1} \
--in2 {input.r2} \
--out1 {output.r1} \
--out2 {output.r2} \
--html {output.html} \
--json {output.json} \
--thread {threads} \
> {log} 2>&1
```

```
"""
```

rule map_and_sort:

```

input:
  reference="data/reference.fasta",
  r1="results/trimmed/demo_R1.fastq.gz",
  r2="results/trimmed/demo_R2.fastq.gz",
output:
  bam="results/mapping/demo.sorted.bam",
log:
  "logs/mapping.log"
threads: 2
resources:
  mem_mb=3000,
  runtime=15,
conda:
  "../envs/mapping.yaml"
shell:
  """
  set -o pipefail
  minimap2 -t {threads} -ax sr {input.reference} {input.r1} {input.r2} 2> {log} \
  | samtools sort -@ {threads} -o {output.bam} - 2>> {log}
  """

```

```

rule index_bam:
  input:
    bam="results/mapping/demo.sorted.bam",
  output:
    bai="results/mapping/demo.sorted.bam.bai",
  threads: 1
  resources:
    mem_mb=1000,
    runtime=5,
  conda:
    "../envs/mapping.yaml"
  shell:
    "samtools index {input.bam} {output.bai}"

```

```

rule flagstat:
  input:
    bam="results/mapping/demo.sorted.bam",
  output:
    report="results/mapping/demo.flagstat.txt",
  threads: 1
  resources:
    mem_mb=1000,
    runtime=5,
  conda:
    "../envs/mapping.yaml"
  shell:
    "samtools flagstat {input.bam} > {output.report}"

```

```

rule multiqc:
  input:
    fastp_html="results/qc/fastp.html",
    fastp_json="results/qc/fastp.json",
    flagstat="results/mapping/demo.flagstat.txt",
  output:
    report="results/multiqc/multiqc_report.html",
  log:
    "logs/multiqc.log"

```



```

threads: 1
resources:
  mem_mb=2000,
  runtime=10,
conda:
  "../envs/multiqc.yaml"
shell:
  """
  multiqc results \
  --force \
  --outdir results/multiqc \
  > {log} 2>&1
  """

```

8.5 Dry-run and lint

```

cd "$RCC_WORK/dna-demo"
conda activate snakemake
snakemake --snakefile workflow/Snakemakefile --dry-run --printshellcmds
snakemake --snakefile workflow/Snakemakefile --lint

```

Snakemake should plan the data-generation, trimming, mapping, indexing, flagstat, and reporting rules.

8.6 Execute through Slurm

```

nice snakemake \
  --snakefile workflow/Snakemakefile \
  --profile RCC_PROFILE \
  --jobs 10 \
  --software-deployment-method conda \
  --rerun-incomplete \
  --printshellcmds

```

The first execution may take longer because Snakemake creates the per-rule environments. Subsequent executions reuse them unless the environment definitions change.

8.7 Inspect the outputs

```

cat results/mapping/demo.flagstat.txt
ls -lh results/mapping/
ls -lh results/qc/
ls -lh results/multiqc/

```

Open these files in VS Code:

results/qc/fastp.html

results/multiqc/multiqc_report.html

An HTML report is a diagnostic artifact, not proof that an analysis is scientifically valid. For real data, inspect read quality, adapter content, mapping rate, duplicate rate, coverage, reference suitability, sample metadata, and contamination controls according to the project protocol.

9. Understanding incremental execution

Why Snakemake does not simply rerun everything

Large workflows may contain hundreds of steps and terabytes of outputs. Repeating successful unaffected work wastes time and shared resources. Snakemake compares requested targets with their dependencies and workflow metadata to decide what must run.

This convenience requires discipline: generated outputs must not be hand-edited, clocks and file systems must behave consistently, and rules must declare all relevant inputs and parameters. Hidden dependencies make incremental execution unreliable.

Run the completed workflow again:

```
snakemake --snakefile workflow/Snakefile --dry-run
```

Snakemake should report that nothing needs to be done.

Now change an input, configuration value, script, or environment file. Snakemake uses file timestamps and its recorded metadata to determine which outputs are outdated. This is why generated results should not be edited manually.

Useful commands include:

```
snakemake --snakefile workflow/Snakefile --summary
snakemake --snakefile workflow/Snakefile --detailed-summary
snakemake --snakefile workflow/Snakefile --list-rules
snakemake --snakefile workflow/Snakefile --dry-run
```

When a job was interrupted and left an incomplete output, retain `--rerun-incomplete` in the execution command.

10. Resource requests and Slurm etiquette

Resources belong to individual rules

Different steps have different bottlenecks. Decompression may use little memory, an aligner may benefit from several threads, and assembly may require substantial memory. Request resources per rule rather than assigning one large request to the entire workflow.

A thread request affects performance only if the program is configured to use those threads. Snakemake's `threads`: value and the command-line option passed to the program must agree. Memory requests protect the node and help scheduling; they do not limit every program automatically unless the site enforces a memory cgroup.

Request resources based on evidence, then refine them.

`threads` is the number of CPU threads the command can actually use.

`mem_mb` is the memory request for one rule job.

`runtime` is the expected wall-clock limit in minutes if the RCC profile uses that convention.

`--jobs` limits the number of simultaneously active or submitted workflow jobs.

Do not request many CPUs for a single-threaded command. Do not request a full node because a program once ran out of memory with no measured usage.

After a representative run, inspect accounting:

```
sacct --starttime today --user "$USER" \
--format=JobID,JobName,State,Elapsed,AllocCPUS,ReqMem,MaxRSS,ExitCode
```

Interpret `MaxRSS` carefully: sites and Slurm versions may report it at job-step granularity. Ask RCC support which accounting fields are reliable for local policy.

Typical failure categories:

OUT_OF_MEMORY - Likely issue: memory request too small or program memory growth; **Correct response:** inspect logs and accounting; increase `mem_mb` based on evidence

TIMEOUT - Likely issue: runtime request too short or command stalled; **Correct response:** inspect log progress; adjust runtime or fix the command

exit code not zero - Likely issue: application or input error; **Correct response:** read the rule log before resubmitting

job pending - **Likely issue:** resources, priority, account, dependency, or policy; **Correct response:** inspect the pending reason; do not repeatedly cancel and resubmit

many tiny jobs - **Likely issue:** workflow granularity too fine; **Correct response:** group or batch work according to RCC guidance

11. Logs and failure handling

Failure is information, not a reason to rerun blindly

A failed workflow may involve several layers: missing input, invalid configuration, environment resolution, scheduler rejection, insufficient resources, program error, or scientific validation. Preserve the first useful error and identify the layer before changing parameters.

Standard output and standard error are text streams. Redirecting them to rule-specific logs provides evidence even after the terminal or tmux session has closed. Slurm accounting provides scheduler-level state and resource information; application logs provide program-level detail. Both are needed.

Every externally executed command should have a predictable log file. A useful log captures:

- command-line output and errors;
- tool version where practical;
- sample or rule identity;
- important parameters; and
- enough context to diagnose a failure without rerunning blindly.

When a workflow stops:

- read the final Snakemake error block;
- identify the failed rule and external Slurm job ID;
- inspect the rule log;
- inspect sacct state and exit code;
- correct the cause;
- run a dry run; and
- restart with `--rerun-incomplete`.

Do not solve an error by deleting the entire results directory unless you understand why every result must be recreated.

12. Reproducibility checklist

Reproducible does not mean merely repeatable on one account

A strong analysis record lets another authorised person understand the input identifiers, obtain or locate the approved data, recreate the software, inspect parameters, execute the workflow, and connect final figures or tables to generated files. Exact byte-for-byte reproduction may still depend on hardware, random seeds, library behaviour, and external databases, so record these sources of variation explicitly.

A project is ready to hand to a colleague when it contains:

- a README.md describing the scientific question and execution command;
- a workflow/Snakefile or modular workflow directory;
- a configuration file with project parameters;
- environment files for every analysis rule;
- immutable input data or a documented, access-controlled source;

sample metadata with stable identifiers;
 logs and final reports;
 a version-control history for code and configuration; and
 no passwords, tokens, private SSH keys, or identifying data committed to Git.
 Record the workflow state:

```
snakemake --version > results/snakemake-version.txt
conda env export --from-history --name snakemake > results/snakemake-control-environment.yml
```

For each Conda environment used by a workflow, the YAML file in envs/ is the primary human-maintained specification. Snakemake records the concrete deployed environment internally.

13. Version control without data leakage

What Git is for

Git records changes to small text-based project assets such as workflow rules, scripts, configuration templates, environment files, and documentation. It supports review, rollback, branches, and pull requests. Git is not a general research-data store and does not remove files from history merely because they were deleted in a later commit.

Before every commit, review the exact files with `git status` and `git diff --staged`. Never commit credentials, private keys, access tokens, patient identifiers, restricted sample sheets, or large generated data.

Initialise Git only for code and small configuration files:

```
git init
```

Create `.gitignore`:

```
# Generated results and logs
results/
logs/
```

```
# Snakemake working state
.snakemake/
```

```
# Large or controlled input data
data/*.fastq
data/*.fastq.gz
data/*.bam
data/*.cram
```

```
# Credentials and local editor state
.env
*.key
*.pem
.vscode/
```

Review before every commit:

```
git status
git diff --staged
```

Never use GitHub or another public service as a transfer mechanism for patient-related, controlled, or large research data.

14. Common beginner mistakes

The underlying pattern in most beginner mistakes is an undeclared assumption: software installed outside the environment, a file created manually, a program run outside Slurm, or a parameter remembered but not stored. The corrective principle is to make inputs, software, parameters, resources, and outputs explicit.

Installing every tool into one environment

This produces dependency conflicts and makes provenance unclear. Keep Snakemake separate and use per-rule environment files.

Running python analyse.py directly

The tutorial script expects the Snakemake context. Execute the requested workflow target, not the script in isolation.

Running Snakemake without the RCC profile

That may execute rule commands on the submission host. Always use the site-supported profile or wrapper for real execution.

Editing generated results

Snakemake may overwrite them, and the edit cannot be reproduced. Change the input, configuration, script, or rule and regenerate the result.

Using absolute paths throughout the Snakefile

Absolute paths make a workflow hard to move. Use a project root and configuration values. Use absolute paths only for stable, documented shared reference resources.

Treating a successful exit code as scientific validation

A technically successful workflow can use the wrong samples, reference, model, or parameters. Reproducibility supports scientific review; it does not replace it.

15. Completion checklist

You have completed Part 2 when all of the following are true:

- ☐ conda --version and mamba --version work in a new terminal.
- ☐ Channel priority is strict.
- ☐ Snakemake is installed in a dedicated control environment.
- ☐ You can start, detach from, and reattach to a tmux session.
- ☐ The statistics workflow passes a dry run and lint check.
- ☐ The statistics rules execute through Slurm, not on the submission host.
- ☐ The statistics workflow creates two TSV files and one PNG figure.
- ☐ The DNA workflow creates per-rule Conda environments automatically.
- ☐ The DNA workflow creates a sorted BAM, BAM index, flagstat report, fastp report, and MultiQC report.
- ☐ Re-running each completed workflow produces “nothing to be done.”
- ☐ You inspected Slurm accounting and rule logs.
- ☐ No private key, password, token, or controlled research data is present in Git.

16. Next steps

After completing Part 2, continue with:

Part 3: performance, resource measurement, storage, compression, and efficient I/O; and

Part 4: Apptainer containers, immutable software images, and containerised Snakemake rules.

Original next steps

After these examples, adapt the pattern rather than copying a command collection. A real project should define:

sample metadata and validation;

reference datasets and versions;

quality-control thresholds;

tool parameters justified by the scientific question;

test data small enough for continuous validation;

a review process for environment updates; and

a retention and publication plan for logs, reports, and results.

Advanced follow-up topics should include modular Snakemake workflows, configuration schemas, unit tests, benchmark directives, job grouping, container execution with Apptainer, reference-data management, workflow reports, and continuous integration with synthetic test data.

References for maintainers

The publication version should link to the official Miniforge, Bioconda, Snakemake, Slurm, and RCC-specific documentation. Version-sensitive commands must be retested whenever the supported RCC software stack changes.